

App Development Project Report

1. Student Details

Name: Sanskar Pandey

Roll Number: 23F3003478

Email: 23f3003478@ds.study.iitm.ac.in

About Me: I am a second-year BS diploma student at IIT Madras, and this is my last Diploma term. I work as a software developer, and I enjoy building web applications that tend to solve problems for a large number of people

2. Project Details

Project Title: Hospital Management System

Problem Statement:

To design and build a web-based hospital management system that enables patients to book appointments, doctors to manage their schedules and patient records, and administrators to oversee hospital operations, including user management, analytics, and reporting.

Approach:

The application was built using Flask as the backend REST API framework with a modular blueprint architecture. Vue.js 3 (via CDN) powers the frontend as a single-page application. The system implements role-based access control for three user types: Admin, Doctor, and Patient. Redis caching improves performance, while Celery handles background tasks like appointment reminders and report generation. The application follows a clean separation of concerns with models, routes, services, and utilities organised in their respective directories.

3. AI/LLM Declaration

I used **Claude (Sonnet Model)** to assist in writing and creating API documentation samples, README, and the extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	Object Relational Mapper for SQLite database
Flask-JWT-Extended	JWT-based authentication and authorisation
Flask-Caching	Redis caching integration
Flask-CORS	Cross-Origin Resource Sharing support
Flask-Mail	Email notification service
Jinja2	Template engine for rendering dynamic HTML pages
Vue.js 3 (CDN)	Frontend SPA framework for dynamic UI
Vue Router 4 (CDN)	Client-side routing for SPA navigation
Bootstrap 5	Frontend styling and responsive design
Chart.js	To visualise trends such as activity vs. time
Flask-Login	User authentication and session management
Redis	The caching layer and the Celery message broker
Celery	Background task processing (reminders, exports)
Celery Beat	Scheduled periodic tasks (daily/monthly jobs)
Werkzeug	Password hashing and security utilities
SQLite	Lightweight local database for storing user data

5. Database Schema / ER Diagram

Tables:

1. **User** - Authentication and role management

Fields: id, username, email, password_hash, role (admin/doctor/patient),

is_active, is_blacklisted, last_login, created_at

2. **Doctor** - Doctor profile and professional information

Fields: id, user_id (FK), department_id (FK), full_name, specialization,
qualification, experience_years, phone, bio, consultation_fee,
is_active, created_at

3. **Patient** - Patient profile and medical information

Fields: id, user_id (FK), full_name, date_of_birth, gender, phone, address,
blood_group, emergency_contact, medical_history, is_active, created_at

4. **Department** - Hospital departments/specializations

Fields: id, name, description, is_active, created_at

5. **Appointment** - Appointment scheduling and management

Fields: id, patient_id (FK), doctor_id (FK), department_id (FK),
appointment_date, appointment_time, duration, status, reason, notes,
cancelled_at, cancelled_by, original_date, created_at

6. **DoctorAvailability** - Doctor working hours schedule

Fields: id, doctor_id (FK), date, start_time_morning, end_time_morning,
start_time_evening, end_time_evening, is_available, slot_duration

7. **Treatment** - Medical records for each appointment

Fields: id, appointment_id (FK), diagnosis, prescription, notes,
tests_recommended, follow_up_date, follow_up_notes, visit_type, created_at

8. **Payment** - Payment transaction records

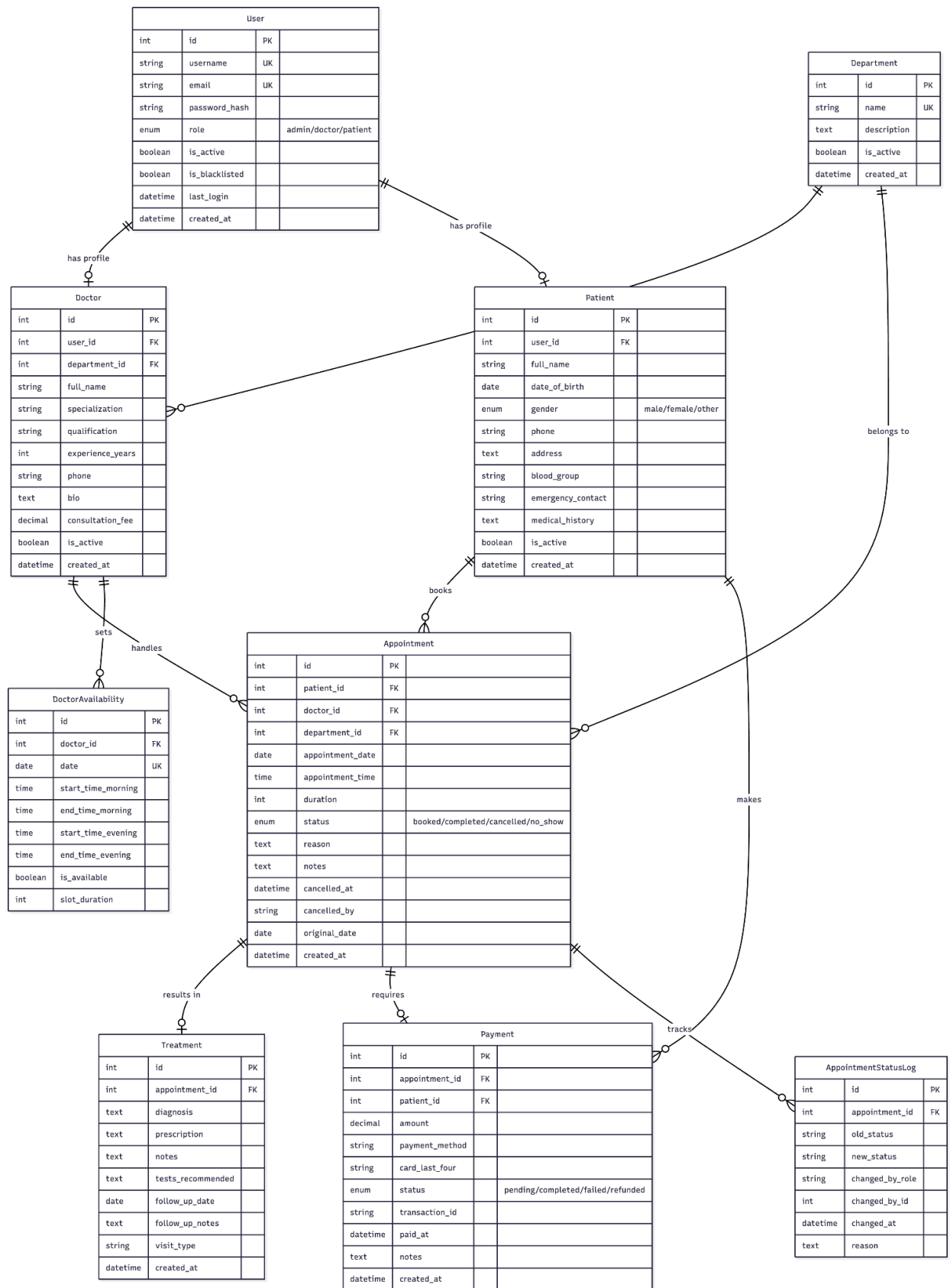
Fields: id, appointment_id (FK), patient_id (FK), amount, payment_method,
card_last_four, status, transaction_id, paid_at, notes, created_at

9. **AppointmentStatusLog** - Audit trail for appointment status changes

Fields: id, appointment_id (FK), old_status, new_status, changed_by_role,
changed_by_id, changed_at, reason

Relationships:

- One-to-One: User -> Patient (user_id)
- One-to-One: User -> Doctor (user_id)
- Many-to-One: Doctor -> Department (department_id)
- Many-to-One: Appointment -> Department (department_id)
- One-to-Many: Doctor -> Appointment (doctor_id)
- One-to-Many: Patient -> Appointment (patient_id)
- One-to-Many: Doctor -> DoctorAvailability (doctor_id)
- One-to-One: Appointment -> Treatment (appointment_id)
- One-to-One: Appointment -> Payment (appointment_id)
- One-to-Many: Appointment -> AppointmentStatusLog (appointment_id)
- One-to-Many: Patient -> Payment (patient_id)



6. API Resource Endpoints

Authentication Endpoints ([/api/auth](#))

Endpoint	Method	Description
/api/auth/register	POST	Register new patient account
/api/auth/login	POST	Authenticate user and get JWT token
/api/auth/logout	POST	Logout and invalidate token
/api/auth/me	GET	Get current user profile
/api/auth/refresh	POST	Refresh JWT access token
/api/auth/change-password	POST	Change user password

Admin Endpoints ([/api/admin](#))

Endpoint	Method	Description
/api/admin/dashboard/stats	GET	Dashboard statistics (cached 2min)
/api/admin/doctors	GET	List all doctors with search/filter
/api/admin/doctors	POST	Create new doctor with user account

<code>/api/admin/doctors/<id></code>	GET	Get doctor details
<code>/api/admin/doctors/<id></code>	PUT	Update doctor information
<code>/api/admin/doctors/<id></code>	DELETE	Deactivate doctor account
<code>/api/admin/doctors/<id>/blacklist</code>	POST	Blacklist/unblacklist doctor
<code>/api/admin/patients</code>	GET	List all patients with search/filter
<code>/api/admin/patients/<id></code>	GET	Get patient details
<code>/api/admin/patients/<id></code>	PUT	Update patient information
<code>/api/admin/patients/<id>/blacklist</code>	POST	Blacklist/unblacklist patient
<code>/api/admin/patients/<id>/appointments</code>	GET	Get patient appointments
<code>/api/admin/patients/<id>/treatments</code>	GET	Get patient treatments
<code>/api/admin/appointments</code>	GET	List all appointments
<code>/api/admin/appointments/<id></code>	GET	Get appointment details

<code>/api/admin/appointments/<id>/cancel</code>	POST	Cancel appointment
<code>/api/admin/appointments/<id>/status-history</code>	GET	Status change history
<code>/api/admin/departments</code>	GET	List all departments
<code>/api/admin/departments</code>	POST	Create new department
<code>/api/admin/departments/<id></code>	PUT	Update department
<code>/api/admin/charts/appointments-trend</code>	GET	Appointments over time chart
<code>/api/admin/charts/appointments-by-department</code>	GET	Department-wise breakdown
<code>/api/admin/charts/appointments-by-status</code>	GET	Status distribution chart
<code>/api/admin/charts/doctor-workload</code>	GET	Doctor workload analysis
<code>/api/admin/charts/patient-registrations</code>	GET	Patient registration trends

<code>/api/admin/charts/revenue</code>	GET	Revenue analytics chart
<code>/api/admin/payments</code>	GET	Admin: List all payments
<code>/api/admin/payments/<id>/refund</code>	POST	Admin: Process refund
<code>/api/admin/payments/stats</code>	GET	Admin: Payment statistics

Doctor Endpoints (`/api/doctor`)

Endpoint	Method	Description
<code>/api/doctor/dashboard/stats</code>	GET	Doctor dashboard statistics
<code>/api/doctor/appointments</code>	GET	List doctor's appointments
<code>/api/doctor/appointments/<id></code>	GET	Get appointment details
<code>/api/doctor/appointments/<id>/complete</code>	POST	Mark appointment as completed
<code>/api/doctor/appointments/<id>/cancel</code>	POST	Cancel appointment

<code>/api/doctor/appointments/<id>/no-show</code>	POST	Mark patient as no-show
<code>/api/doctor/patients</code>	GET	List doctor's patients
<code>/api/doctor/patients/<id></code>	GET	Get patient details
<code>/api/doctor/patients/<id>/history</code>	GET	Get patient medical history
<code>/api/doctor/treatments/<id></code>	GET	Get treatment details
<code>/api/doctor/treatments/<id></code>	PUT	Update treatment record
<code>/api/doctor/availability</code>	GET	Get availability schedule
<code>/api/doctor/availability</code>	POST	Set availability for a date
<code>/api/doctor/availability/bulk</code>	POST	Set bulk availability (multiple dates)
<code>/api/doctor/profile</code>	GET	Get doctor profile
<code>/api/doctor/profile</code>	PUT	Update doctor profile

<code>/api/doctor/charts/weekly-appointments</code>	GET	Weekly appointment chart
<code>/api/doctor/charts/appointments-by-status</code>	GET	Status breakdown chart
<code>/api/doctor/charts/patient-visits</code>	GET	Patient visit trends
<code>/api/doctor/charts/monthly-summary</code>	GET	Monthly summary chart

7. Architecture and Features (optional)

Architecture Overview:

Backend Structure:

- **backend/app.py** - Flask application entry point with `create_app` factory
- **backend/app/__init__.py** - Application factory and blueprint registration
- **backend/app/models/** - SQLAlchemy database models (9 models)
- **backend/app/routes/** - Flask Blueprints for API endpoints (6 blueprints)
- **backend/app/services/** - Business logic services (cache, notification, validation)
- **backend/app/tasks/** - Celery background tasks (reminders, reports, exports)
- **backend/app/utils/** - Utility functions and decorators
- **backend/app/templates/** - HTML templates for reports and emails
- **backend/celery_app.py** - Celery configuration with beat schedule
- **backend/config.py** - Configuration classes (Development, Production, Testing)
- **backend/init_db.py** - Database initialisation and seeding script

Frontend Structure:

- **frontend/index.html** - SPA entry point with CDN imports
- **frontend/src/main.js** - Vue app initialization
- **frontend/src/router/index.js** - Vue Router with 40+ routes
- **frontend/src/components/** - Reusable Vue components (Navbar)

- **frontend/src/views/** - Page components organized by role
- - **admin/** - 6 admin views (Dashboard, DoctorManagement, etc.)
- - **doctor/** - 6 doctor views (Dashboard, Appointments, etc.)
- - **patient/** - 10 patient views (Dashboard, Booking, etc.)
- **frontend/src/services/** - API service modules (auth, admin, doctor, patient)
- **frontend/manifest.json** - PWA manifest configuration
- **frontend/sw.js** - Service worker for offline support

Implemented Features (Core):

1. User Authentication and Authorisation

- JWT-based authentication with access tokens (24hr expiry)
- Role-based access control (Admin, Doctor, Patient)
- Login, registration (patients only), logout
- Password change functionality
- User blacklisting capability

2. Admin Dashboard and Management

- Real-time statistics dashboard (doctors, patients, appointments, revenue)
- Doctor CRUD operations with department assignment
- Patient management with profile editing
- Appointment oversight and cancellation
- User blacklist/activation controls
- Department management (create, update)

3. Doctor Dashboard and Features

- Personal dashboard with today's appointments
- Weekly schedule view
- Appointment management (complete, cancel, mark no-show)
- Treatment record creation and updates

- Patient history access
- Availability scheduling (7-day, morning/evening slots)
- Profile management

4. Patient Dashboard and Booking

- Department browsing with doctor listings
- Doctor search with filters (department, name, qualification)
- Available slot viewing
- Appointment booking with reason
- Appointment cancellation and rescheduling
- Medical history and treatment viewing
- Profile management

5. Appointment System

- Conflict prevention (unique doctor-date-time constraint)
- Status workflow (booked -> completed/cancelled/no_show)
- Status change logging with audit trail
- Appointment rescheduling support

6. Payment System

- Multiple payment methods (Card, Cash, Insurance)
- Payment processing (simulated)

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1dBIQ_RQ7i4dMxoox0GH8tXXxCPmde3ss/view?usp=sharing
